

Cybersecurity Wargame Internship Task Report

LAB 1: KRYPTON – <https://overthewire.org/wargames/krypton/>

This level has 7 levels.

Level 1: Krypton level 1 is accessed through the command

```
ssh -p 2231 krypton1@krypton.labs.overthewire.org
```

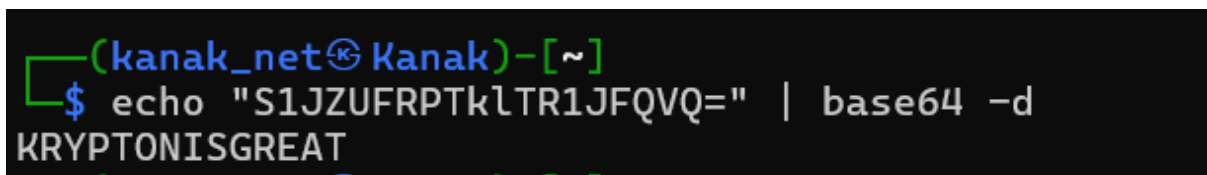
Password of level 1 is encrypted in algorithm of Base64 is **S1JZUFRPTk1TR1JFQVQ=**

This password is decrypted using the following command

```
echo "S1JZUFRPTk1TR1JFQVQ=" | base64 -d
```

This got the decrypted password as KRYPTONISGREAT.

Using this password, I had a breakthrough to level 1.



```
(kanak_net ☿ Kanak) - [~]  
$ echo "S1JZUFRPTk1TR1JFQVQ=" | base64 -d  
KRYPTONISGREAT
```

Level 2: Krypton level 2 is accessed through the command

```
ssh -p 2231 krypton2@krypton.labs.overthewire.org
```

The password for level 2 is in the file 'krypton2'. It is 'encrypted' using a simple rotation. It is also in non-standard ciphertext format. When using alpha characters for cipher text it is normal to group the letters into 5 letter clusters, regardless of word boundaries. This helps obfuscate any patterns. This file has kept the plain text word boundaries and carried them to the cipher text

Krypton2 file is accessed through the command

```
cd /krypton/krypton2
```

```
Cat krypton2
```

Content of krypton2 is YRIRY GJB CNFFJBEQ EBGGRA

It is decrypted using the command

```
Cat krypton2 | tr "[A-Z]" "[M-AZ-L]"
```

Decrypted ciphertext is LEVEL TWO PASSWORD ROTTEN.

Through this password ROTTEN got me to walk through level 2.

```

krypton1@bandit:~$ cd /krypton/krypton1
krypton1@bandit:/krypton/krypton1$ ls
krypton2  README
krypton1@bandit:/krypton/krypton1$ cat README
Welcome to Krypton!

This game is intended to give hands on experience with cryptography
and cryptanalysis. The levels progress from classic ciphers, to modern,
easy to harder.

Although there are excellent public tools, like cryptool, to perform
the simple analysis, we strongly encourage you to try and do these
without them for now. We will use them in later excercises.

** Please try these levels without cryptool first **

The first level is easy. The password for level 2 is in the file
'krypton2'. It is 'encrypted' using a simple rotation called ROT13.
It is also in non-standard ciphertext format. When using alpha characters for
cipher text it is normal to group the letters into 5 letter clusters,
regardless of word boundaries. This helps obfuscate any patterns.

This file has kept the plain text word boundaries and carried them to
the cipher text.

Enjoy!
krypton1@bandit:/krypton/krypton1$ cat krypton2
YRIRY GJB CNFFJBEQ EBGGRA

```

```

(kanak_net@Kanak)-[~]
$ tr "[ABCDEFGHIJKLMNOPQRSTUVWXYZ]" "[NOPQRSTUVWXYZABCDEFGHIJKLM]"
YRIRY GJB CNFFJBEQ EBGGRA
LEVEL TWO PASSWORD ROTTEN

```

Level 3: Krypton level 3 is accessed through the command

`ssh -p 2231 krypton3@krypton.labs.overthewire.org`

Description: ROT13 is a simple substitution cipher.

Substitution ciphers are a simple replacement algorithm. In this example of a substitution cipher, we will explore a 'monoalphabetic' cipher. Monoalphabetic means, literally, "one alphabet" and you will see why.

This level contains an old form of cipher called a 'Caesar Cipher'. A Caesar cipher shifts the alphabet by a set number. For example:

plain: a b c d e f g h i j k ...

cipher: G H I J K L M N O P Q ...

In this example, the letter 'a' in plaintext is replaced by a 'G' in the ciphertext so, for example, the plaintext 'bad' becomes 'HGJ' in ciphertext.

The password for level 3 is in the file krypton3. It is in 5 letter group ciphertext. It is encrypted with a Caesar Cipher. Without any further information, this cipher text may be difficult to break. You do not have direct access to the key, however you do have access to a program that will encrypt anything you wish to give it using the key. If you think logically, this is completely easy.

One shot can solve it!

Have fun.

Additional Information:

The encrypt binary will look for the keyfile in your current working directory. Therefore, it might be best to create a working directory in /tmp and in there a link to the keyfile. As the encrypt binary runs setuid krypton3, you also need to give krypton3 access to your working directory.

Here is an example:

```
krypton2@melinda:~$ mktemp -d
/tmp/tmp.Wf2OnCpCDQ
krypton2@melinda:~$ cd /tmp/tmp.Wf2OnCpCDQ
krypton2@melinda:/tmp/tmp.Wf2OnCpCDQ$ ln -s /krypton/krypton2/keyfile.dat
krypton2@melinda:/tmp/tmp.Wf2OnCpCDQ$ ls
keyfile.dat
krypton2@melinda:/tmp/tmp.Wf2OnCpCDQ$ chmod 777 .
krypton2@melinda:/tmp/tmp.Wf2OnCpCDQ$ /krypton/krypton2/encrypt /etc/issue
krypton2@melinda:/tmp/tmp.Wf2OnCpCDQ$ ls
ciphertext keyfile.dat
```

Password to level 3 is in krypton 3 encrypted through the encrypt binary script in the level 2.

To analyse the password pattern a plain text file is created using command

Touch ptext

Nano ptext

Inserted alphabets from A to Z and encrypted them using the given binary encrypt that's how I got the pattern

Then translat

ed given encrypted text to plain text using the following command

```
cat /krypton/krypton2/krypton3 | tr "[MNOPQRSTUVWXYZABCDEFGHIJKL]"
"[ABCDEFGHIJKLMNPOQRSTUVWXYZ]"
```

Password for level 3 turned out to be CAESARISEASY.

Through this password I got walk through level 3

```

krypton2@bandit:/krypton/krypton2$ mktemp -d
/tmp/tmp.udSdcBjm6g
krypton2@bandit:/krypton/krypton2$ cd /tmp/tmp.udSdcBjm6g
krypton2@bandit:/tmp/tmp.udSdcBjm6g$ ln -s /krypton/krypton2/keyfile.dat
krypton2@bandit:/tmp/tmp.udSdcBjm6g$ ls
keyfile.dat
krypton2@bandit:/tmp/tmp.udSdcBjm6g$ chmod 777 .
krypton2@bandit:/tmp/tmp.udSdcBjm6g$ /krypton/krypton2/encrypt /etc/issue
krypton2@bandit:/tmp/tmp.udSdcBjm6g$ ls
ciphertext keyfile.dat
krypton2@bandit:/tmp/tmp.udSdcBjm6g$ cat /etc/issue
Ubuntu 24.04.2 LTS \n \l

krypton2@bandit:/tmp/tmp.udSdcBjm6g$ cat ciphertext
GNGZFGXFEZX
krypton2@bandit:/tmp/tmp.udSdcBjm6g$ touch ptext
krypton2@bandit:/tmp/tmp.udSdcBjm6g$ nano ptext
Unable to create directory /home/krypton2/.local/share/nano/: No such file or directory
It is required for saving/loading search history or cursor positions.

krypton2@bandit:/tmp/tmp.udSdcBjm6g$ nano ptest
Unable to create directory /home/krypton2/.local/share/nano/: No such file or directory
It is required for saving/loading search history or cursor positions.

krypton2@bandit:/tmp/tmp.udSdcBjm6g$ nano ptext
Unable to create directory /home/krypton2/.local/share/nano/: No such file or directory
It is required for saving/loading search history or cursor positions.

krypton2@bandit:/tmp/tmp.udSdcBjm6g$ cat ptext
ABCDEFGHIJKLMNOPQRSTUVWXYZ

krypton2@bandit:/tmp/tmp.udSdcBjm6g$ /krypton/krypton2/encrypt ptext
krypton2@bandit:/tmp/tmp.udSdcBjm6g$ ls
ciphertext keyfile.dat ptext
krypton2@bandit:/tmp/tmp.udSdcBjm6g$ cat ciphertext
MNOPQRSTUVWXYZABCDEFGHIJKL
krypton2@bandit:/tmp/tmp.udSdcBjm6g$ GNGZFGXFEZX | tr "[A-z]" "[M-A-Z-L]"
tr: range-endpoints of 'M-A' are in reverse collating sequence order
GNGZFGXFEZX: command not found
krypton2@bandit:/tmp/tmp.udSdcBjm6g$ cat /krypton/krypton2/krypton3
OMQEMDUEQMEK

krypton2@bandit:/tmp/tmp.udSdcBjm6g$ cat /krypton/krypton2/krypton3 | tr "[MNOPQRSTUVWXYZABCDEFGHIJKL]" "[ABCDEFGHIJKLMNOPQRSTUVWXYZ]"
CAESARISEASY

```

Level 4: Krypton level 4 is accessed through the command
<ssh -p 2231 krypton4@krypton.labs.overthewire.org>

Description:

Well done. You've moved past an easy substitution cipher.

The main weakness of a simple substitution cipher is repeated use of a simple key. In the previous exercise you were able to introduce arbitrary plaintext to expose the key. In this example, the cipher mechanism is not available to you, the attacker.

However, you have been lucky. You have intercepted more than one message. The password to the next level is found in the file 'krypton4'. You have also found 3 other files. (found1, found2, found3)

You know the following important details:

The message plaintexts are in American English (** very important) - They were produced from the same key (** even better!)

Text given in found1, found2, found3 is analysed through following commands:

```

python3 freq_analysis.py /krypton/krypton3/found1 1
python3 freq_analysis.py /krypton/krypton3/found2 1
python3 freq_analysis.py /krypton/krypton3/found3 1

```

Frequency of the letter occurring is analysed through the frequently occurring letter in actual english language English after which there is a use of mindful guesses. To decrypt the code Which was "KSVVW BGSJD SVSIS VXBMN YQUUK BNWCU ANMJS" through the following commands:

```
cat krypton4 | tr "JDSQBKVIWGMNXUYC]" "[THEAOWLVDNURFSP]"
```

Decrypted text is WELLD ONETH ELEVE LFOUR PASSW ORDIS ARUTE.

Now the password for level 4 is guessed to be BRUTE.

```
krypton3@bandit:/tmp/tmp.FYhD62n8gG$ cat krypton4 | tr "JDS]" "[THE]"
krypton3@bandit:/tmp/tmp.FYhD62n8gG$ NMTE krypton3@bandit:/tmp/mp.FYhD62n8
krypton3@bandit:/tmp/tmp.FYhD62n8gG$ cat krypton4 | tr "JDSQBKV]" "[THEAOWL]"
krypton3@bandit:/tmp/tmp.FYhD62n8gG$ cat krypton4 | tr "JDSQBKVIW]" "[THE
AOWLVDN]"
WELLD ONETH ELEVE LXOMN YAUUW ONDCU A
krypton3@bandit:/tmp/tmp.FYhD62n8gG$ cat krypton4 | tr "JDSQBKVIWGMN]" "[T
HEAOWLVDNUR]"
WELLD ONETH ELEVE LXOUR YAUUW ORDCU ARUTE krypton3@bandit:/tmp/tmp.FYhD6
tr: missing operand after '[JDSQBKVIWGMN]'
Two strings must be given when translating.
Try 'tr --help' for more information.
krypton3@bandit:/tmp/tmp.FYhD62n8gG$ cat krypton4 | tr "JDSQBKVIWGMNX]" "[T
HEAOWLVDNURF]"
OHWWV ADHT
HWWLH WRANU YEUUO AUVCU AUNTH krypton3@bandit:/tmp/tmp.FYhD62n8gG$
krypton3@bandit:/tmp/tmp.FYhD62n8gG$
krypton3@bandit:/tmp/tmp.FYhD62n8gG$ cat krypton4 | tr "JDSQBKVIWGMNX]" "[
THEAOWLVDNURF]"
WELLD ONETH ELEVE LFOUR YAUUW ORDCU ARUTE krypton3@bandit:/tmp/tmp.FYhD62n8
gG$ cat krypton4 | tr "JDSQBKVIWGMNX]" "[THEAOWLVDNURF]"
WELLD ONETH ELEVE LFOUR YAUUW ORDCU ARUTE
cat: k: No such file or directory
krypton3@bandit:/tmp/tmp.FYhD62n8gG$ cat krypton4 | tr "JDSQBKVIWGMNXUY]"
"[THEAOWLVDNURFSP]"
krypton3@bandit:/tmp/tmp.FYhD62n8gG$ cat krypton4 | tr "JDSQBKVIWGMNXUYc]"
"[THEAOWLVDNURFI]"
tr: missing operand after '[JDSQBKVIWGMNXUYc]'
Two strings must be given when translating.
Try 'tr --help' for more information.
[THEAOWLVDNURFI]: command not found
krypton3@bandit:/tmp/tmp.FYhD62n8gG$
krypton3@bandit:/tmp/tmp.FYhD62n8gG$
krypton3@bandit:/tmp/tmp.FYhD62n8gG$ cat krypton4 | tr "JDSQBKVIWGMNXUYC]"
"[THEAOWLVDNURFSP]"
WELLD ONETH ELEVE LFOUR PASSW ORDIS ARUTE krypton3@bandit:/tmp/tmp.FYhD62n8
Connection to krypton.labs.overthewire.org closed.
```

Level 5: Krypton level 5 is accessed through the command

```
ssh -p 2231 krypton5@krypton.labs.overthewire.org
```

Description:

You more than likely used some form of FA and some common sense to solve that one.

So far we have worked with simple substitution ciphers. They have also been

‘monoalphabetic’, meaning using a fixed key, and giving a one to one mapping of plaintext (P) to ciphertext (C). Another type of substitution cipher is referred to as ‘polyalphabetic’, where one character of P may map to many, or all, possible ciphertext characters.

An example of a polyalphabetic cipher is called a Vigenère Cipher. It works like this:

If we use the key(K) ‘GOLD’, and P = PROCEED MEETING AS AGREED, then “add” P to K, we get C. When adding, if we exceed 25, then we roll to 0 (modulo 26).

```
P P R O C E E D M E E T I N G A S A G R E E D \
K G O L D G O L D G O L D G O L D G O L D G O \
```

becomes:

```
P 15 17 14 2 4 4 3 12 4 4 19 8 13 6 0 18 0 6 17 4 4 3 \
K 6 14 11 3 6 14 11 3 6 14 11 3 6 14 11 3 6 14 11 3 6 14 \
C 21 5 25 5 10 18 14 15 10 18 4 11 19 20 11 21 6 20 2 8 10 17 \
```

So, we get a ciphertext of:

VFZFK SOPKS ELTUL VGUCH KR

This level is a Vigenère Cipher. You have intercepted two longer, english language messages (American English). You also have a key piece of information. You know the key length!

For this exercise, the key length is 6. The password to level five is in the usual place, encrypted with the 6 letter key.

Following commands are executed to decode the password

```
/tmp/tmp.MUz2M4WQdh$
krypton4@bandit:/krypton/krypton4$ cd /tmp/tmp.MUz2M4WQdh$
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ ls
ls: command not found
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ ls
freq_analysis.py keylength.py vignere_decoder.py vignere_shift.py
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ python3 vignere_shift.py /krypton/krypton4/found1 6 0
YIYWNQRLYTRHYDJTWZSLNNHTMJJYFNYYJJSJLWNMFXXBBXIXIMJTBMIYJJNTYBWKWMLFGWISJSZSYSPJNFJQFWTYWKJMMNSYWKYSAYMTSQZJRFDKMXFJJPKFSTTTJH
IMZYFSJWJNTWGWJYGZTMTYSFFJTWJQBSFSJSJIJKNKWSXZYKXMSIIFTXSSKTJTYWMYKLTISNJFITWIXNBSJHJF
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ python3 freq_analysis.py /krypton/kryp
ton4/found1_shift0 1
No file named '/krypton/krypton4/found1_shift0'
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ python3 freq_analysis.py found1_shift0
1
No file named 'found1_shift0'
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ python3 vignere_shift.py /krypton/kryp
ton4/found1 6 0> found1_shift0
YIYWNQRLYTRHYDJTWZSLNNHTMJJYFNYYJJSJLWNMFXXBBXIXIMJTBMIYJJNTYBWKWMLFGWISJSZSYSPJNFJQFWTYWKJMMNSYWKYSAYMTSQZJRFDKMXFJJPKFSTTTJH
IMZYFSJWJNTWGWJYGZTMTYSFFJTWJQBSFSJSJIJKNKWSXZYKXMSIIFTXSSKTJTYWMYKLTISNJFITWIXNBSJHJF
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ python3 freq_analysis.py found1_shift0 1
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ python3 freq_analysis.py found1_shift0 1
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ python3 freq_analysis.py found1_shift0 1
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ ls
found1_shift0 freq_analysis.py keylength.py vignere_decoder.py vignere_shift.py
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ python3 freq_analysis.py found1_shift0 1
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ ls
found1_shift0 freq_analysis.py keylength.py vignere_decoder.py vignere_shift.py
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ python3 vignere_shift.py /krypton/krypton4/found1 6 1> found1_shift1
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ ls
found1_shift0 found1_shift1 freq_analysis.py keylength.py vignere_decoder.py vignere_shift.py
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ python3 freq_analysis.py found1_shift1 1
1>
27

krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ python3 vignere_shift.py /krypton/krypton4/found1 6 2> found1_shift2
YIYWNQRLYTRHYDJTWZSLNNHTMJJYFNYYJJSJLWNMFXXBBXIXIMJTBMIYJJNTYBWKWMLFGWISJSZSYSPJNFJQFWTYWKJMMNSYWKYSAYMTSQZJRFDKMXFJJPKFSTTTJHBMJDSQIJPJTSJ
IMZYFSJWJNTWGWJYGZTMTYSFFJTWJQBSFSJSJIJKNKWSXZYKXMSIIFTXSSKTJTYWMYKLTISNJFITWIXNBSJHJF
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ python3 freq_analysis.py found1_shift2 1
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ python3 vignere_shift.py /krypton/krypton4/found1 6 2> found1_shift2
YIYWNQRLYTRHYDJTWZSLNNHTMJJYFNYYJJSJLWNMFXXBBXIXIMJTBMIYJJNTYBWKWMLFGWISJSZSYSPJNFJQFWTYWKJMMNSYWKYSAYMTSQZJRFDKMXFJJPKFSTTTJHBMJDSQIJPJTSJ
IMZYFSJWJNTWGWJYGZTMTYSFFJTWJQBSFSJSJIJKNKWSXZYKXMSIIFTXSSKTJTYWMYKLTISNJFITWIXNBSJHJF
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ ls
found1_shift0 found1_shift1 found1_shift2 freq_analysis.py keylength.py vignere_decoder.py vignere_shift.py
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ python3 freq_analysis.py found1_shift2
Usage: python3 freq_analysis.py filename groupsize
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ python3 freq_analysis.py found1_shift2 1
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ python3 vignere_shift.py /krypton/krypton4/found1 6 3> found1_shift3
YIYWNQRLYTRHYDJTWZSLNNHTMJJYFNYYJJSJLWNMFXXBBXIXIMJTBMIYJJNTYBWKWMLFGWISJSZSYSPJNFJQFWTYWKJMMNSYWKYSAYMTSQZJRFDKMXFJJPKFSTTTJHBMJDSQIJPJTSJ
IMZYFSJWJNTWGWJYGZTMTYSFFJTWJQBSFSJSJIJKNKWSXZYKXMSIIFTXSSKTJTYWMYKLTISNJFITWIXNBSJHJF
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ python3 freq_analysis.py found1_shift3 1
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$
krypton4@bandit:/tmp/tmp.MUz2M4WQdh$ python3 vignere_decoder.py /krypton/krypton4/krypton5 FREKEY
Decoding file '/krypton/krypton4/krypton5' with key 'FREKEY':
CLEARTXT
```

Password to level 5 turned out to be CLEARTXT.

Level 6: Krypton level 6 is accessed through the command
`ssh -p 2231 krypton6@krypton.labs.overthewire.org`

Description:

FA can break a known key length as well. Lets try one last polyalphabetic cipher, but this time the key length is unknown. Note: the text is written in American English
Enjoy.

```

krypton5@bandit:~$ cd /krypton/krypton5
krypton5@bandit:/krypton/krypton5$ mkdir -p /tmp/tmp.jQKZTDQeyZ
krypton5@bandit:/krypton/krypton5$ cd /tmp/tmp.jQKZTDQeyZ
krypton5@bandit:/tmp/tmp.jQKZTDQeyZ$ ls
keylength.py
krypton5@bandit:/tmp/tmp.jQKZTDQeyZ$ ls
keylength.py
krypton5@bandit:/tmp/tmp.jQKZTDQeyZ$ LS
LS: command not found
krypton5@bandit:/tmp/tmp.jQKZTDQeyZ$ ls
freq_analysis.py  keylength.py  vignere_decoder.py  vignere_shift.py
krypton5@bandit:/tmp/tmp.jQKZTDQeyZ$ python3 keylength.py /krypton/krypton5/found1
Chance key length is:
1 = 14.82%
2 = 13.64%
3 = 12.92%
4 = 6.12%
5 = 2.04%
6 = 12.92%
7 = 1.96%
8 = 2.6%
9 = 9.03%
10 = 1.82%
11 = 1.01%
12 = 5.84%
13 = 0.87%
14 = 1.68%
15 = 1.73%
16 = 0.89%
17 = 0.42%
18 = 9.03%

```

```

A: 1
krypton5@bandit:/tmp/tmp.jQKZTDQeyZ$ python3 vignere_decoder.py
Usage: python3 vignere_decoder.py filename key
krypton5@bandit:/tmp/tmp.jQKZTDQeyZ$ python3 vignere_decoder.py /krypton/krypton5 KEYLENGTH
Decoding file '/krypton/krypton5' with key 'KEYLENGTH':

No file named '/krypton/krypton5'
krypton5@bandit:/tmp/tmp.jQKZTDQeyZ$ python3 vignere_decoder.py /krypton/krypton5/krypton6 KEYLENGTH
Decoding file '/krypton/krypton5/krypton6' with key 'KEYLENGTH':

RANDOM
krypton5@bandit:/tmp/tmp.jQKZTDQeyZ$ python3 vignere_decoder.py /krypton/krytimed out waiting for input: auto-logout
Connection to krypton.labs.overthewire.org closed.
PS C:\Users\c2sha>

```

Password of level 6 was RANDOM

Level 7: Krypton level 7 is accessed through the command

`ssh -p 2231 krypton7@krypton.labs.overthewire.org`

Description:

A feature of good crypto is random ciphertext. A good cipher must not reveal any clues about the plaintext. Since natural language plaintext (in this case, English) contains patterns, it is left up to the encryption key or the encryption algorithm to add the 'randomness'.

Modern ciphers are similar to older plain substitution ciphers, but improve the 'random' nature of the key.

An example of an older cipher using a complex, random, large key is a vignere using a key of the same size of the plaintext. For example, imagine you and your confidence have agreed on a key using the book 'A Tale of Two Cities' as your key, in 256 byte blocks.

The cipher works as such:

Each plaintext message is broken into 256 byte blocks. For each block of plaintext, a corresponding 256 byte block from the book is used as the key, starting from the first chapter, and progressing. No part of the book is ever re-used as key. The use of a key of the same length as the plaintext, and only using it once is called a "One Time Pad".

Look in the krypton6 directory. You will find a file called 'plain1', a 256 byte block. You will also see a file 'key1', the first 256 bytes of 'A Tale of Two Cities'. The file 'cipher1' is the cipher text of plain1. As you can see (and try) it is very difficult to break the cipher without the key knowledge.

If the encryption is truly random letters, and only used once, then it is impossible to break. A truly random "One Time Pad" key cannot be broken. Consider intercepting a ciphertext message of 1000 bytes. One could brute force for the key, but due to the random key nature, you would produce every single valid 1000 letter plaintext as well. Who is to know which is the real plaintext?!?

Choosing keys that are the same size as the plaintext is impractical. Therefore, other methods must be used to obscure ciphertext against frequency analysis in a simple substitution cipher. The impracticality of an 'infinite' key means that the randomness, or entropy, of the encryption is introduced via the method.

We have seen the method of 'substitution'. Even in modern crypto, substitution is a valid technique. Another technique is 'transposition', or swapping of bytes.

Modern ciphers break into two types; symmetric and asymmetric.

Symmetric ciphers come in two flavours: block and stream.

Until now, we have been playing with classical ciphers, approximating 'block' ciphers. A block cipher is done in fixed size blocks (surprise!). For example, in the previous paragraphs we discussed breaking text and keys into 256 byte blocks, and working on those blocks. Block ciphers use a fixed key to perform substitution and transposition ciphers on each block discretely.

Its time to employ a stream cipher. A stream cipher attempts to create an on-the-fly 'random' keystream to encrypt the incoming plaintext one byte at a time. Typically, the 'random' key byte is xor'd with the plaintext to produce the ciphertext. If the random keystream can be replicated at the receiving end, then a further xor will produce the plaintext once again.

From this example forward, we will be working with bytes, not ASCII text, so a hex editor/dumper like hexdump is a necessity. Now is the right time to start to learn to use tools like [cryptool](#).

In this example, the keyfile is in your directory, however it is not readable by you. The binary 'encrypt6' is also available. It will read the keyfile and encrypt any message you desire, using the key AND a 'random' number. You get to perform a 'known ciphertext' attack by introducing plaintext of your choice. The challenge here is not simple, but the 'random' number generator is weak.

As stated, it is now that we suggest you begin to use public tools, like cryptool, to help in your analysis. You will most likely need a hint to get going. See 'HINT1' if you need a kickstart.

If you have further difficulty, there is a hint in 'HINT2'.

The password for level 7 (krypton7) is encrypted with 'encrypt6'.


```
cypher1 key1 plain1
krypton6@bandit:/krypton/krypton6/onetime$ mktemp -d
/tmp/tmp.n3QWpGBmYN
krypton6@bandit:/krypton/krypton6/onetime$ cd /tmp/tmp.n3QWpGBmYN
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ ln -s /krypton/krypton6/keyfile.dat
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ ls
keyfile.dat
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ chmod 777 .
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ ls /krypton/krypton6
encrypt6 HINT1 HINT2 keyfile.dat krypton7 onetime README
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ touch tale.txt
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ nano
Unable to create directory /home/krypton6/.local/share/nano/: No such file or directory
It is required for saving/loading search history or cursor positions.

krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ NANO
NANO: command not found
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ nano
Unable to create directory /home/krypton6/.local/share/nano/: No such file or directory
It is required for saving/loading search history or cursor positions.

krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ nano tale.txt
Unable to create directory /home/krypton6/.local/share/nano/: No such file or directory
It is required for saving/loading search history or cursor positions.

krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ nano tale.txt
Unable to create directory /home/krypton6/.local/share/nano/: No such file or directory
It is required for saving/loading search history or cursor positions.

krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ cat tale.txt
ITWASTHEBESTOFTHETIMESITWASTHEWORSTOFTHETIMESx
```

```
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ /krypton/krypton6/encrypt6 tale.txt ciphertale
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ ls
ciphertale keyfile.dat tale.txt
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ cat ciphertale
MBYTVZFmZDCMVSPLVYFVGHNNAOUDYRAMLTLWUDBFDDBTRKFkrypton6@bandit:/tmp/tmp.n3QWpGBmYN$ xxd -b tale.txt
00000000: 01001001 01010100 01010111 01000001 01010011 01010100 ITWAST
00000006: 01001000 01000101 01000010 01000101 01010011 01010100 HEBEST
0000000c: 01001111 01000110 01010100 01001000 01000101 01010100 OFTHET
00000012: 01001001 01001101 01000101 01010011 01001001 01010100 TWESIT
00000018: 01010111 01000001 01010011 01010100 01001000 01000101 WASTHE
0000001e: 01010111 01001111 01010010 01010011 01010100 01001111 WORSTO
00000024: 01000110 01010100 01001000 01000101 01010100 01001001 FTHETI
0000002a: 01001101 01000101 01010011 01111000 00001010 MESx.
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ xxd -b ciphertale
00000000: 01001101 01000010 01011001 01010100 01010110 01011010 MBYTVZ
00000006: 01000110 01001101 01011010 01000100 01000011 01001101 FMZDCM
0000000c: 01010110 01010011 01001100 01010000 01010110 01011001 VSLPVY
00000012: 01000110 01001011 01000111 01001000 01001110 01001110 FKGHNN
00000018: 01000001 01001111 01010101 01000100 01011001 01010010 AOUDYR
0000001e: 01000001 01010111 01010100 01001100 01010111 01010101 AWTLMU
00000024: 01000100 01000010 01000110 01000100 01000100 01000010 DBFDDB
0000002a: 01010100 01010010 01001011 01000110 TRKF
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ python -c "print('A'*100)"
Command 'python' not found, did you mean:
  command 'python3' from deb python3
  command 'python' from deb python-is-python3
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ python -c "print('A'*100)"
Command 'python' not found, did you mean:
  command 'python3' from deb python3
  command 'python' from deb python-is-python3
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ python-is-python3 -c "print('A'*100)"
python-is-python3: command not found
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ python3 -c "print('A'*100)"
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ python3 -c "print('A'*100)" > a.txt
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ cat a.txt
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ /krypton/krypton6/encrypt6 a.txt cipher_a.txt
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ cat cipher_a.txt
EICTDGYIZKTHNSIRFXCYCFUEOCKRNEICTDGYIZKTHNSIRFXCYCFUEOCKRNEICTDGYIYZkrypton6@bandit:/tmp/tmp.n3QWpGBmYN$ python3 vignere_dec
```

```
EICTDGYIZKTHNSIRFXCYCFUEOCKRNEICTDGYIZKTHNSIRFXCYCFUEOCKRNEICTDGYIYZKTHNSIRFXCYCFUEOCKRNEICTDGYIYZkrypton6@bandit:/tmp/tmp.n3QWpGBmYN$ python3 vignere_dec
oder.py /krypton/krypton6/krypton7 EICTDGYIZKTHNSIRFXCYCFUEOCKRN
Usage: python3 vignere_decoder.py filename key
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ ls
a.txt cipher_a.txt ciphertale keyfile.dat tale.txt vignere_decoder.py
krypton6@bandit:/tmp/tmp.n3QWpGBmYN$ python3 vignere_decoder.py /krypton/kry
pton6/krypton7 EICTDGYIZKTHNSIRFXCYCFUEOCKRN
Decoding file '/krypton/krypton6/krypton7' with key 'EICTDGYIZKTHNSIRFXCYCFUEOCKRN':
LFSRISNOTRANDOM
```